Four YOLO Small Detectors on a 204-Photograph Custom Dataset: What a 21-Image Test Set Can and Cannot Tell Us

Tanjim Samad Swapnil

Mechatronics Engineering, Rajshahi University of Engineering & Technology (RUET), Bangladesh

10 November 2026

Abstract

Small custom datasets are normal in industrial inspection, yet results on them are often read as if the test split were large. I trained YOLOv8s, YOLO11s, YOLO12s and YOLO26s with one fixed recipe (COCO-pretrained weights, AdamW, batch size 4, 640 px, online augmentation switched off) on 489 Roboflow-augmented images that come from 163 photographs of a packaged-medicine carton labelled *Good* or *Miss_Print*, and evaluated them once on 21 further photographs holding 29 boxes. Test mAP@50:95 ranged from 0.583 (YOLOv8s) to 0.727 (YOLO12s); mAP@50 ranged from 0.858 to 0.960. An image-level bootstrap put 95 % intervals about ±0.1 wide around every mAP@50:95. YOLO12s and YOLO26s could not be separated (difference 0.035, interval −0.022 to +0.099), nor could YOLOv8s and YOLO11s, whereas both newer models exceeded both older ones. Training box loss fell from about 1.5 to 0.11–0.29 while validation box loss never fell below 0.95 and reached 1.5–2.1 at its worst, and the best checkpoint came as early as epoch 8 of 33. Two of the four models lost 0.09–0.12 of validation mAP@50:95 within 25 epochs after their peak, the other two 0.02–0.04. The evidence supports a hypothesis that the two newer architectures overfit less on this dataset; it does not establish a ranking. The report explains why, lists what was not tested, and gives a prioritised plan to turn the hypothesis into a result. All code, logs, figures and weights are in the accompanying repository.

1 Introduction

A bounding-box detector for a production-line check is often trained on a few hundred photographs taken by the person who will deploy it. The data are expensive in time rather than money, the defect class is rare or hand-made, and a published benchmark for the exact product does not exist. Two habits then cause trouble. The first is to report a single mAP from a handful of test images with four decimals, which suggests a precision the sample cannot support. The second is to treat falling training loss as progress.

This project compares four recent members of the Ultralytics YOLO family on one such dataset. The question was practical: which of YOLOv8s, YOLO11s, YOLO12s and YOLO26s should be used for a two-class carton-print inspection task? The answer turned out to depend less on the architectures than on how little the data can distinguish between them. The contributions are therefore methodological and descriptive:

- an audited description of the dataset, showing that 489 training files represent 163 independent photographs and that validation and test contain 20 and 21 photographs with no photograph shared between splits (Section 3);

- a four-way comparison under an identical recipe, with per-class results, training dynamics and an operating-point error count (Section 5);
- image-level bootstrap intervals and paired model differences for the test metrics, which show which parts of the ranking survive resampling and which do not (Section 5.5);
- an evidence-linked explanation of why a dataset of this size fails to support firm conclusions and why the detectors overfit it (Section 6), followed by what was not tested and what to do next (Sections 7–8).

No claim is made that these four models are ranked correctly in general; the experiment has one training run per model and one product.

2 Background

2.1 The four detectors

All four are single-stage, anchor-free detectors from the Ultralytics code base [1], descended from the original YOLO formulation [2]. Table 1 lists the published facts that matter for this comparison. YOLO11 replaces the C2f blocks of YOLOv8 with C3k2 blocks and adds a spatial-attention module (C2PSA) to the backbone [3]. YOLOv12 centres the network on attention: area attention, a residual aggregation block (R-ELAN), FlashAttention, no positional encoding and a 7×7 separable convolution as position perceiver; its authors report that FlashAttention limits it to recent NVIDIA GPU generations [4]. YOLO26 removes distribution focal loss, offers an NMS-free one-to-one head, trains with a hybrid SGD–Muon optimiser (MuSGD) and adds ProgLoss and a small-target-aware label assignment (STAL) [5, 6]. The only independent account of YOLO26 that I found is itself a secondary review of Ultralytics material [6], so its architectural description rests on the vendor's documentation.

Model	Params (M)	FLOPs (B)	COCO $\text{mAP}_{50:95}$	Source of the three numbers
YOLOv8s	11.2	28.6	44.9	Ultralytics docs [3]
YOLO11s	9.4	21.6	47.0	Ultralytics docs [3]
YOLOv12-S	9.3	21.4	48.0	original paper [4]
YOLO26s	9.5	20.9	48.6	Ultralytics docs [5]

Table 1: Published COCO figures at 640 px for the four checkpoints. The rows come from different documents and evaluation set-ups, so they should not be subtracted from each other. The notebook's model summaries list about 11.14, 9.43, 9.25 and 9.95 M parameters for the unfused models (the last figure, for YOLO26s, is above the documented 9.5 M; a 9.47 M summary also appears, probably the fused model, and I did not trace this further).

2.2 Why small datasets fail

Four well-documented effects apply directly here. (i) *Memorisation is cheap.* Networks of this class can fit random labels on training images [7], so a low training loss does not show that anything general was learned. (ii) *Deep networks rely on data volume to avoid overfitting*, and augmentation is the standard remedy [8]. (iii) *Pretraining helps but is not a regulariser.* On COCO subsets, training from scratch broke down between roughly 3.5k and 10k images, whereas pretrained models still worked at 1k images (9.9 versus 3.5 AP); the same paper reports that pretraining overfit on a 35k-image subset under unchanged hyper-parameters and states that pretraining does not necessarily provide regularisation [9]. (iv) *Test-set size limits what a benchmark can resolve.* Variance from data sampling is well explained by the limited statistical power of the test set, and

the authors recommend resampling over a single fixed held-out set [10]. Ultralytics' own guidance, written for YOLOv5 but still the vendor's training advice, asks for at least 1 500 images and 10 000 labelled instances per class [11]. This project has 371 and 306 training instances.

3 Dataset and audit

The images were photographed and annotated with bounding boxes in Roboflow (project `Fault_Detection_Model` version 2). Four exports of the same project (YOLOv8, YOLO11, YOLO12 and YOLO26 formats) were downloaded; SHA-256 hashes of every image and label file were identical across the four, so one copy was used (1 060 files: 530 images and 530 label files). A validation script rejected unknown class ids, coordinates outside $[0, 1]$ and degenerate boxes, and converted six polygon labels in the training split to enclosing boxes. Every image is 640×640 pixels.

Split	Photographs*	Images	Boxes	Good / Miss_Print
Train	163	489	677	371 / 306
Valid	20	20	27	15 / 12
Test	21	21	29	14 / 15

Table 2: Dataset composition. *Counted from the file-name stem before Roboflow's `.rf.` hash: 163 distinct stems in training with at most three files each, and one file per stem in validation and test. No stem occurs in two splits and no byte-identical image occurs in two splits (checked by MD5).

Two facts from this audit shape everything that follows. First, the training split is a threefold offline augmentation of 163 photographs (489 = 3×163), so the effective number of independent training scenes is 163, not 489. Roboflow documents that offline augmentation is applied to training images only, which matches what I found [12]; sampled training images show rotation with black borders and speckle noise. Second, the validation and test splits are tiny: a single missed box changes recall by 3.7 points on validation and 3.4 on test. About 0.16 of the image area is covered by the median box (0.019 to 0.96 range), so objects are large and mostly isolated; 8 of 21 test images contain two boxes.

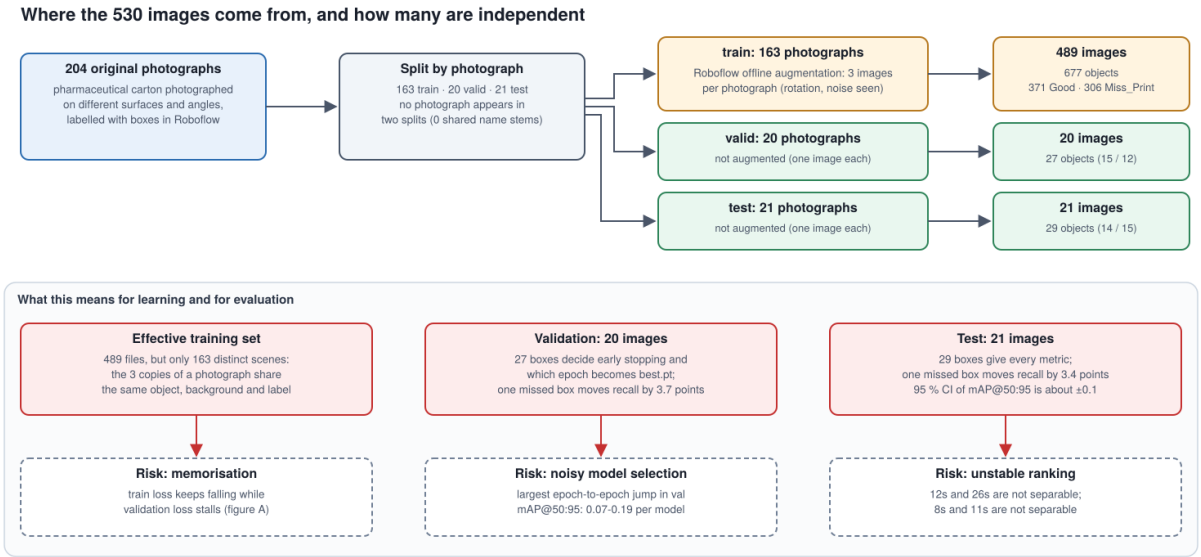


Figure 1: Where the 530 images come from and what the split sizes imply. Photograph counts are inferred from file names.

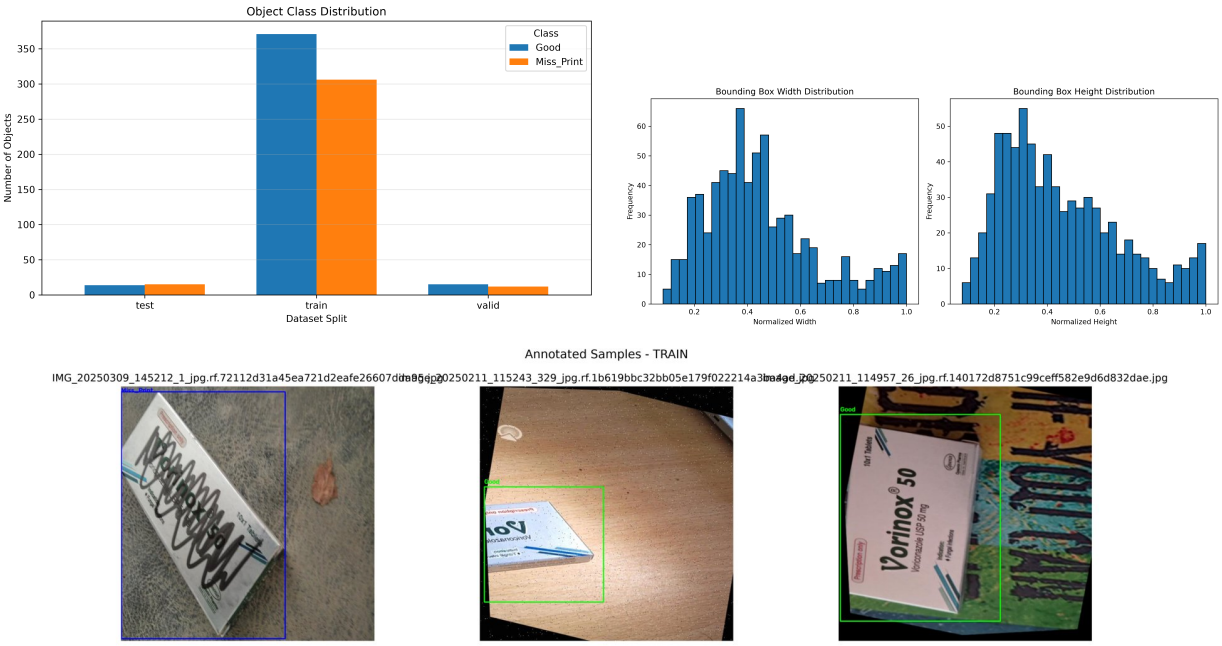


Figure 2: Top: object counts per class and split; normalised box width and height. Bottom: three annotated training images (one *Miss_Print*, two *Good*) showing the rotation, black borders and speckle noise introduced by offline augmentation.

4 Experimental set-up

Recipe. Each model started from its COCO-pretrained small checkpoint (`yolov8s.pt`, `yolo11s.pt`, `yolo12s.pt`, `yolo26s.pt`) and was fine-tuned with AdamW (learning rate 0.001, weight decay 0.0005), batch size 4, image size 640, seed 42, deterministic mode, up to 100 epochs with early stopping at patience 25. All online augmentation was disabled (mosaic, mixup, copy-paste, HSV, flips, scale, translate, rotation, shear and perspective set to zero), so the only augmentation seen is the offline Roboflow copies. The four `args.yaml` files differ only in the model name and run directory. No hyper-parameter search was performed and there is one run per model. Full settings are in Appendix A.

Hardware and software. Training used an NVIDIA GeForce RTX 5060 Laptop GPU (7.9 GB), Python 3.11.3, PyTorch 2.11.0+cu128 and Ultralytics 8.4.190 on Windows 10. The test re-evaluation and bootstrap were run again on CPU in FP32 with the same Ultralytics version.

Checkpoint choice and evaluation. Ultralytics keeps `best.pt` by validation fitness, which for detection weights mAP@50 by 0.1 and mAP@50:95 by 0.9. The checkpoint was evaluated once on the test split with `model.val`. Precision and recall are the class means at the confidence that maximises the mean F1 curve; the F1 in the tables is the harmonic mean of those two means, as defined in the notebook. mAP@50 and mAP@50:95 follow the COCO-style averaging over IoU thresholds [13]. My CPU re-evaluation reproduced all logged test numbers to four decimals except YOLO26s mAP@50:95, which was 0.6925 against 0.6969 logged on the GPU; I report the logged value in tables and the CPU value inside the bootstrap intervals, and the difference is a reminder of how small numerical changes move a 29-box metric.

Bootstrap. To express test-sampling uncertainty I resampled the 21 test images with replacement 2 000 times (seed 42) and recomputed precision, recall, mAP@50 and mAP@50:95 with Ultralytics' `ap_per_class` on the validator's own per-image statistics. The same resamples were used for all four models, so differences between models are paired. Intervals are 2.5th–97.5th percentiles. This captures the variance that comes from which images are in the test set; it does not capture variance between training runs or seeds, which was not measured.

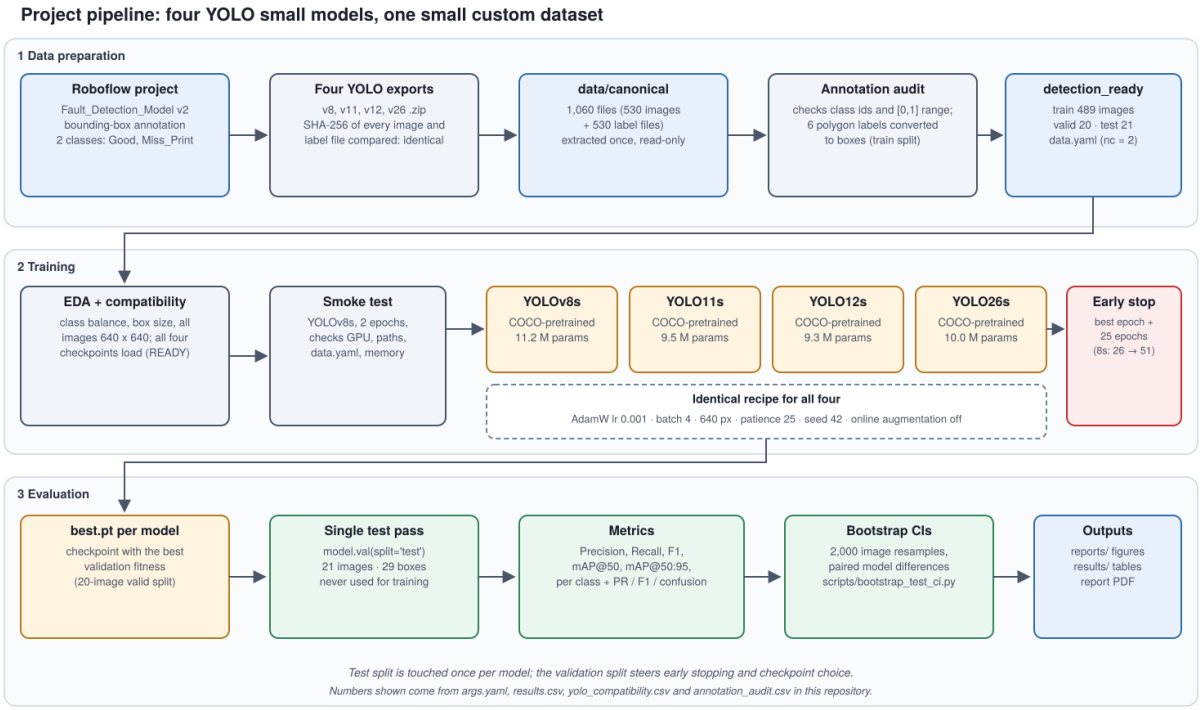


Figure 3: End-to-end pipeline of the project.

5 Results

5.1 Training dynamics

Table 3 and Figure 4 show the same pattern for all four models: training losses decrease steadily, whereas validation losses fall for a few epochs and then fluctuate. The best checkpoint came at epoch 26 of 51 (YOLOv8s), 8 of 33 (YOLO11s), 47 of 72 (YOLO12s) and 39 of 64 (YOLO26s); because patience was 25, each run stopped exactly 25 epochs after its best. Validation box loss in YOLOv8s was lowest at epoch 12 (1.10) and ended at 1.31; in YOLO11s it was lowest at epoch 22 (1.04) and ended at 1.46. YOLO12s and YOLO26s ended close to their minimum (1.03 versus 0.95; 1.09 versus 1.00). In validation mAP@50:95 the same split appears: the 25 patience epochs cost YOLOv8s 0.087 and YOLO11s 0.123, but YOLO12s only 0.042 and YOLO26s 0.020.

Model	Epochs run	Best epoch	Best val $mAP_{50:95}$	Last val $mAP_{50:95}$	Change	Val box loss min → last	s/epoch
YOLOv8s	51	26	0.654	0.567	−0.087	1.10 → 1.31	8.0
YOLO11s	33	8	0.661	0.538	−0.123	1.04 → 1.46	9.2
YOLO12s	72	47	0.699	0.657	−0.042	0.95 → 1.03	12.0
YOLO26s	64	39	0.729	0.709	−0.020	1.00 → 1.09	12.4

Table 3: Training summary from `results.csv`. "Last" is the final epoch before early stopping, i.e. 25 epochs after the best. Seconds per epoch include validation and come from a single run on a laptop GPU; they are not a controlled speed benchmark.

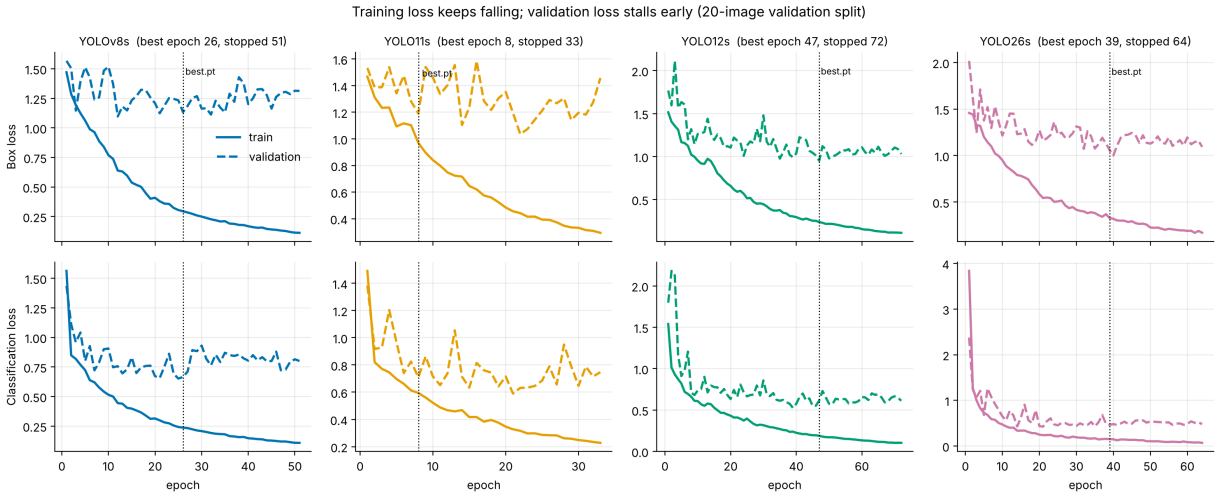


Figure 4: Training (solid) and validation (dashed) box and classification loss for each model; the dotted line marks the epoch kept as `best.pt`. The validation split has 20 images.

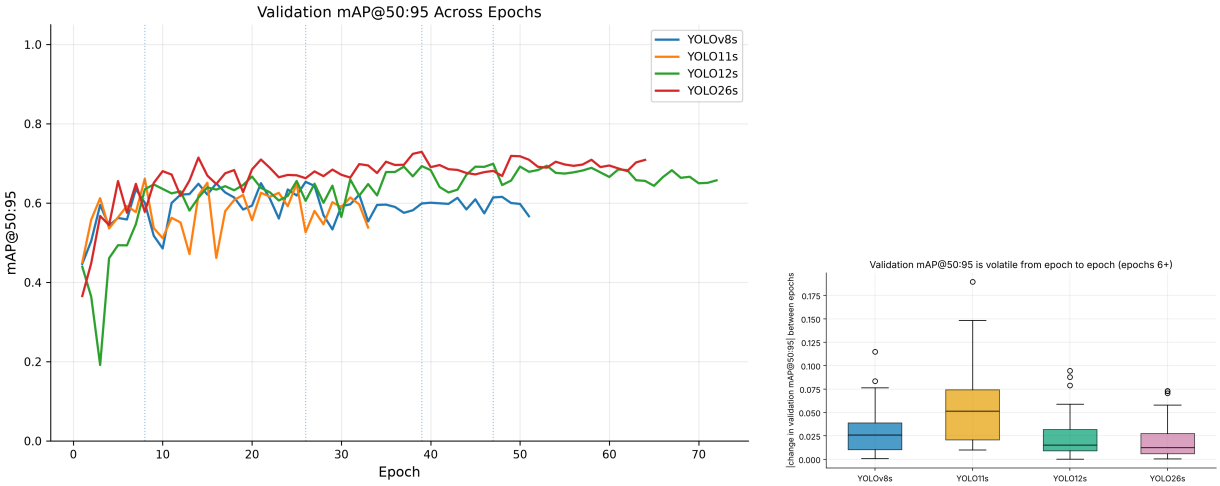


Figure 5: Left: validation mAP@50:95 per epoch (dotted lines mark each model's best epoch). Right: absolute epoch-to-epoch change after the first five epochs. Median changes were 0.026, 0.051, 0.015 and 0.012 and the largest jumps 0.115, 0.190, 0.095 and 0.073 for YOLOv8s, YOLO11s, YOLO12s and YOLO26s.

5.2 Test performance

Table 4 gives the single-pass test results. By point estimate the order on mAP@50:95 is YOLO12s (0.727), YOLO26s (0.697), YOLO11s (0.625), YOLOv8s (0.583). YOLO12s reaches precision 1.000 on test, which only means that no detection above the F1-optimal threshold was a false positive on 29 boxes.

Model	Precision	Recall	F1	mAP@50	mAP@50:95
YOLOv8s	0.8910	0.8534	0.8718	0.8577	0.5834
YOLO11s	0.8535	0.8772	0.8652	0.9066	0.6246
YOLO12s	**1.0000**	**0.9577**	**0.9784**	**0.9600**	**0.7271**
YOLO26s	0.9492	0.8952	0.9214	0.9188	0.6969

Table 4: Test-split results (21 images, 29 boxes), values as logged by the notebook. Bold marks the highest point estimate; Section 5.5 shows it is not always distinguishable from the runner-up.

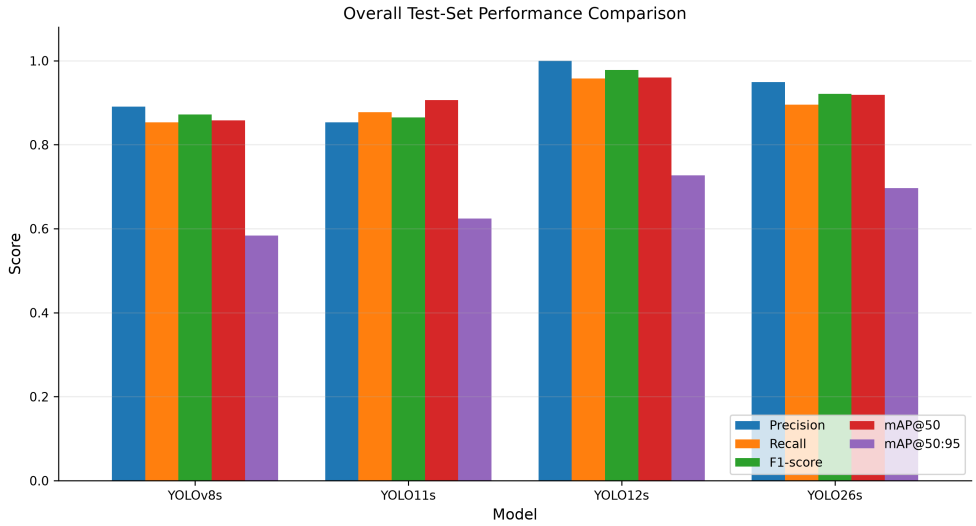


Figure 6: Test metrics for the four models (point estimates; see Figure 7 for uncertainty).

5.3 Per-class results

All four models score lower on *Good* than on *Miss_Print* in mAP@50:95 (Table 5); the largest gap is YOLO26s (0.599 versus 0.795). Each class has only 14 or 15 test boxes, so per-class gaps of this size are within the sampling noise shown in Section 5.5 and should not be read as a property of the classes.

Model	Class	Precision	Recall	F1	mAP@50	mAP@50:95
YOLOv8s	Good	0.7819	0.7857	0.7838	0.7803	0.5083
	Miss_Print	1.0000	0.9211	0.9589	0.9350	0.6585
YOLO11s	Good	0.8365	0.8571	0.8467	0.8920	0.5830
	Miss_Print	0.8705	0.8972	0.8837	0.9213	0.6662
YOLO12s	Good	1.0000	0.9258	0.9615	0.9250	0.7105
	Miss_Print	1.0000	0.9896	0.9948	0.9950	0.7438
YOLO26s	Good	0.9218	0.8571	0.8883	0.8496	0.5991
	Miss_Print	0.9767	0.9333	0.9545	0.9879	0.7946

Table 5: Per-class test results (14 Good and 15 Miss_Print boxes).

5.4 Operating-point errors

Metrics such as mAP hide how many boxes are involved. Table 6 counts object-level outcomes at confidence $\geq$0.25 and IoU $\geq$0.5 with class-aware matching, computed from the same validator statistics. The three best-scoring configurations all miss 3 of 29 boxes. YOLOv8s misses 6, YOLO11s has 6 false positives, and YOLO12s has none. Each of those counts is a handful of objects, which is the reason the intervals below are wide.

Model	True positives	False positives	False negatives (of 29)
YOLOv8s	23	3	6
YOLO11s	26	6	3
YOLO12s	26	0	3
YOLO26s	26	1	3

Table 6: Object-level counts on the test split at confidence ≥0.25 and IoU 0.5.

5.5 Uncertainty

Table 7 and Figure 7 give 95 % bootstrap intervals. For mAP@50:95 they span 0.18 (YOLO12s) to 0.28 (YOLOv8s), i.e. roughly ±0.09 to ±0.14, and the bootstrap standard deviation is 0.046–0.073. A plain binomial argument gives the same order of magnitude: for an accuracy-like proportion $\hat{p}$ measured on $n = 29$ independent objects, $\mathrm{SE} = \sqrt{\hat{p}(1-\hat{p})/n}$, which is 0.056 at $\hat{p} = 0.9$ and a 95 % half-width of about 0.11. Objects in the same image are not independent, so the true uncertainty is, if anything, larger.

Model	Precision	Recall	mAP@50	mAP@50:95
YOLOv8s	0.891 [0.777, 0.993]	0.853 [0.667, 0.967]	0.858 [0.663, 0.990]	0.583 [0.429, 0.713]
YOLO11s	0.854 [0.769, 0.973]	0.877 [0.720, 1.000]	0.907 [0.755, 0.993]	0.625 [0.502, 0.720]
YOLO12s	1.000 [0.952, 1.000]	0.958 [0.810, 1.000]	0.960 [0.880, 0.995]	0.727 [0.632, 0.807]
YOLO26s	0.949 [0.903, 0.996]	0.895 [0.733, 1.000]	0.919 [0.802, 0.995]	0.693 [0.596, 0.774]

Table 7: Test point estimate with 95 % percentile bootstrap interval (2 000 resamples of 21 images). Points are from the CPU re-evaluation; they equal the logged values except YOLO26s mAP@50:95 (0.693 versus 0.697).



Figure 7: Test mAP@50 and mAP@50:95 with 95 % bootstrap intervals.

Table 8 answers the question that matters for model choice. On mAP@50:95, the newer pair (YOLO12s, YOLO26s) beats the older pair (YOLOv8s, YOLO11s) in all four cross-comparisons with intervals that exclude zero, by 0.07–0.14. Inside each pair the models are not separable: YOLO12s minus YOLO26s is +0.035 (interval −0.022 to +0.099; YOLO12s ahead in 87 % of resamples), and YOLOv8s minus YOLO11s is −0.038 (−0.131 to +0.033). On mAP@50 only YOLO12s separates from YOLOv8s and YOLO11s; its lead over YOLO26s has an interval whose lower bound is zero (0.000 to 0.126).

Comparison (A − B)	Mean difference	95 % interval	Excludes 0
YOLOv8s − YOLO11s	−0.038	[−0.131, 0.033]	no
YOLOv8s − YOLO12s	−0.145	[−0.246, −0.059]	yes
YOLOv8s − YOLO26s	−0.109	[−0.203, −0.027]	yes
YOLO11s − YOLO12s	−0.106	[−0.181, −0.039]	yes
YOLO11s − YOLO26s	−0.071	[−0.127, −0.020]	yes
YOLO12s − YOLO26s	+0.035	[−0.022, 0.099]	no

Table 8: Paired bootstrap differences in test mAP@50:95 (same 2 000 resamples for both models).

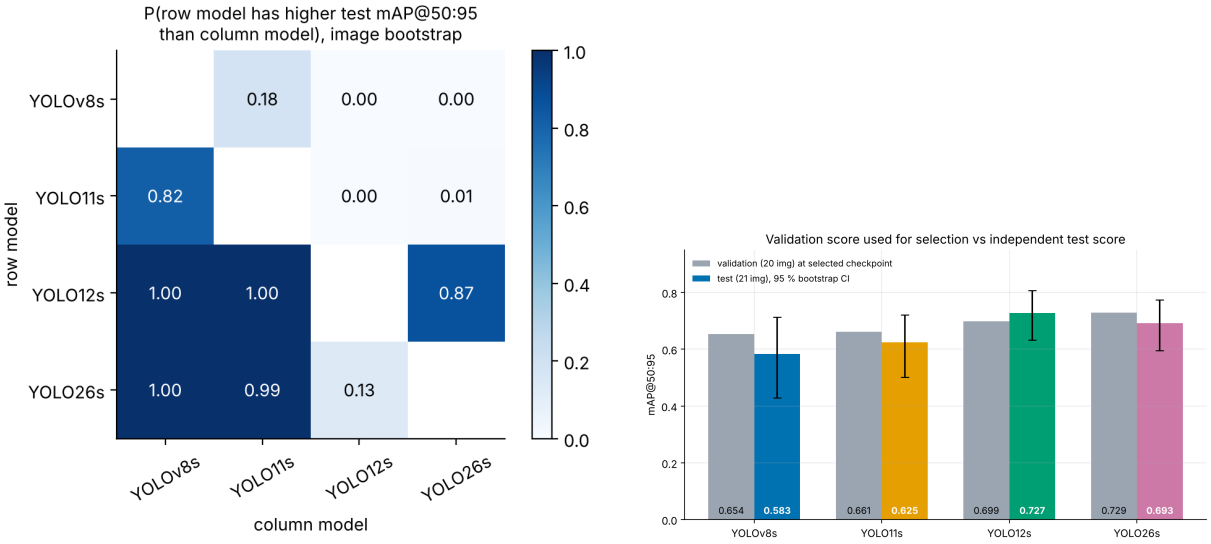


Figure 8: Left: share of bootstrap resamples in which the row model has the higher test mAP@50:95. Right: validation mAP@50:95 of the selected checkpoint against the test value with its interval (test points from the CPU re-evaluation).

What the interval does and does not cover. The test images are the same for all models, and each model has one training run. A separation that survives resampling therefore says that the gap is not an accident of which 21 photographs were chosen. It does not say that the architecture caused the gap, because a different seed or a different early-stopping epoch could move a model by an unknown amount.

5.6 Validation versus test

The checkpoint was chosen on the 20-image validation split, so validation scores are expected to be optimistic. The data are consistent with that for three models and not for the fourth: test mAP@50:95 was lower than validation by 0.071 (YOLOv8s), 0.036 (YOLO11s) and 0.032 (YOLO26s, logged value) and higher by 0.028 (YOLO12s). With four models and intervals of ±0.1 this is not a measurement of selection bias, only a reminder that validation numbers cannot be used as performance claims.

6 Why the small dataset fails and why the models overfit

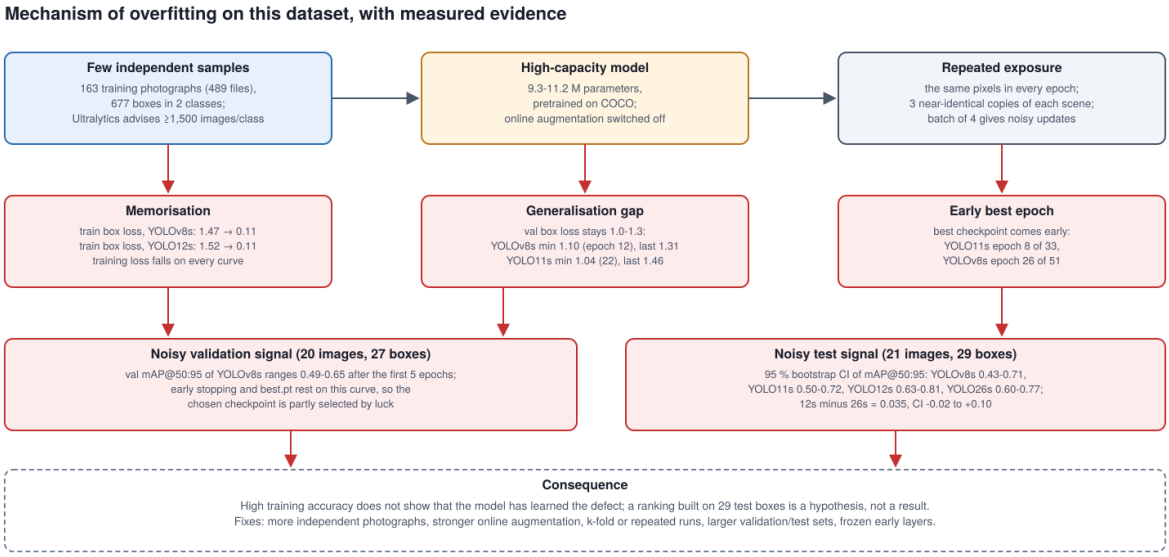


Figure 9: Causal chain from the data and training choices to the observed symptoms, with the measured evidence in each box.

Few independent samples. The training split contains 677 boxes from 163 photographs, a few percent of the vendor's guideline of 10 000 instances per class [11]. The two extra copies of each photograph are rotated or noisy versions of the same carton on the same background, so they add robustness to rotation and noise, not new scenes or new defects. Augmentation is known to help against overfitting [8], but offline copies of one scene are only a limited form of it.

High capacity and repeated exposure. The detectors have 9–11 million parameters; networks of this class can memorise random labels [7], so memorising 163 scenes is easy. With online augmentation set to zero (a choice made to keep the four runs identical), every epoch showed the model the same pixels. Training box loss fell from about 1.5 to 0.11 (YOLOv8s, YOLO12s), 0.17 (YOLO26s) and 0.29 (YOLO11s), while validation box loss never went below 0.95 and, after epoch 5, stayed between 0.95 and 1.6 (Figure 4). Pretraining kept the starting point good (validation mAP@50 of 0.74–0.83 after the first epoch for every model), but it did not stop the validation signal from flattening within 8 to 47 epochs, in line with the finding that pretraining is not a regulariser [9]. Batch size 4 additionally makes each update noisy and batch-norm statistics unreliable; I did not test this.

Noisy selection. Early stopping and `best.pt` depend on validation fitness measured on 27 boxes. Epoch-to-epoch jumps of 0.07–0.19 in validation mAP@50:95 (Figure 5) are larger than most differences between epochs that the procedure tries to rank, so the selected epoch is partly chance. The same split is used both to stop and to choose, which is why its scores are optimistic.

Noisy evaluation. With 29 test boxes each missed object costs 3.4 recall points, and the high IoU thresholds in mAP@50:95 depend on a few well-fitted boxes. The bootstrap intervals in Table 7 are the practical expression of that limit, and a numerically tiny change (GPU against CPU arithmetic) moved YOLO26s mAP@50:95 by 0.004. The statistical point is the one made for benchmarks generally: the test set size can be the limiting factor [10].

Why the newer models looked less overfit here. The two models that lost least after their validation peak (YOLO12s, YOLO26s) are also the two that win on test mAP@50:95. This ordering is suggestive, not established: it rests on four runs, and I did not vary the seed. Plausible reasons that I did not test include the attention blocks and residual scaling of YOLOv12 [4], the

optimiser and loss changes of YOLO26 [5], or simply the longer runs that put the best epoch later in training.

High mAP@50, lower mAP@50:95. Test mAP@50 of 0.86–0.96 against mAP@50:95 of 0.58–0.73 means the detectors find the carton but place boxes less tightly at strict IoU. With large, isolated objects (median box covering 16 % of the image) that gap says more about box-tightness conventions and label precision than about recognition of the defect, and it is the metric most affected by annotation noise.

7 Threats to validity and limitations

- **One run per model, one seed.** Training variance is unknown, so no architectural ranking can be claimed.
- **Test set of 21 photographs and 29 boxes.** Intervals are about ±0.1; percentile bootstrap intervals are themselves approximate at this size and are skewed for metrics bounded near 1.
- **Validation reused for stopping and checkpoint choice.** Validation scores are optimistic; only the test numbers are independent of that choice, and they were evaluated once.
- **Online augmentation disabled.** This removes the main regulariser for all four models, so absolute scores are probably lower than a tuned recipe would give. Whether the ranking would survive default augmentation was not tested, and no augmented baseline was run.
- **No from-scratch or frozen-backbone baseline.** I cannot say how much of the performance comes from COCO pretraining.
- **One product and one defect family.** Sampled images show the same carton type with dark scribble-like marks on *Miss_Print* examples. Generalisation to other products, print defects, lighting or cameras is untested, and the report cannot say whether the 204 photographs cover the intended deployment.
- **Photograph count is inferred** from file-name stems, not from the original Roboflow project.
- **Speed was not benchmarked.** Seconds per epoch (Table 3) are from single runs on one laptop GPU; no inference-latency or edge-device test was made.
- **Documentation sources.** The YOLO26 description relies on vendor documentation and on one secondary review; the Ultralytics dataset-size guidance is a heuristic from the YOLOv5 pages.

8 What to do next

In order of expected value per hour of work:

1. **Collect more independent photographs, not more augmented copies.** Add new cartons, surfaces, lighting and defect instances. Grow validation and test to at least 100 images each so a single object moves recall by under one point.
2. **Re-split by photograph with cross-validation.** Five-fold cross-validation over the 204 photographs (augmenting only inside the training folds) uses every photograph for testing once and gives a variance estimate per model.
3. **Repeat each run with at least three to five seeds**, report mean and spread, and only then compare architectures.

4. **Switch online augmentation on** (mosaic, scale, HSV, flips where physically valid) and compare against the off setting for at least one model; consider freezing early backbone layers for the very smallest data.
5. **Use a held-out set the selection never touches** and report intervals with every metric, as done here.
6. **Measure what the deployment needs**: inference latency on the target device, false-positive rate on defect-free frames, and robustness to lighting and viewpoint.

9 Conclusion

On 21 test photographs, YOLO12s had the highest point estimates (mAP@50 0.960, mAP@50:95 0.727), followed by YOLO26s, YOLO11s and YOLOv8s. After resampling, only a coarse statement survives: the two newer models score higher than the two older ones on mAP@50:95, and the order within each pair is not resolved. The same models also lost the least validation accuracy after their peak, which points to milder overfitting but rests on one run each. The dataset itself, 163 independent training photographs and 20 and 21 for validation and test, is too small to rank four similar detectors and too small for validation scores to be trusted. Adding independent photographs and repeating runs would change this far more than switching architecture.

Reproducibility

The repository contains the notebook, the exact training arguments, per-epoch logs, best checkpoints, all result tables, the bootstrap script (`scripts/bootstrap_test_ci.py`) and a leakage-audit script. The dataset images are not redistributed; `data/README.md` explains how to rebuild them from the Roboflow project.

Items for the author to confirm before publishing

- The labelling rule that separates *Good* from *Miss_Print*, who annotated, and how many products and cameras are in the data.
- The number of original photographs and the augmentation recipe in the Roboflow project (here inferred from file names and sample images).
- Whether the dataset may be shared publicly, and under which licence.
- AI-assistance disclosure: this report was drafted with Claude (Anthropic) from the author's notebook, logs and files; every number was computed from those files or from the re-evaluation described above, and should be checked by the author.

A Training settings

Setting	Value (identical for all four models)
Task / data	detect; `data/detection_ready/data.yaml` (nc = 2)
Epochs / patience	100 / 25
Batch / image size	4 / 640
Optimiser	AdamW, lr0 0.001, lrf 0.01, momentum 0.937, weight decay 0.0005
Warm-up	3 epochs (momentum 0.8, bias lr 0.1)
Loss gains	box 7.5, cls 0.5, dfl 1.5 (not used by YOLO26)
Augmentation	mosaic, mixup, copy-paste, HSV, flips, scale, translate, degrees, shear, perspective all 0
Other	seed 42, deterministic, AMP on, workers 2, save_period 10, close_mosaic 10
Pretrained weights	`yolov8s.pt`, `yolo11s.pt`, `yolo12s.pt`, `yolo26s.pt`

Table 9: Settings read from the four `args.yaml` files, which differ only in model and directory names. In the Ultralytics release used here the `auto_augment` and `erasing` entries are listed in the files but apply to classification only according to the notebook's configuration.

B Additional test-set figures

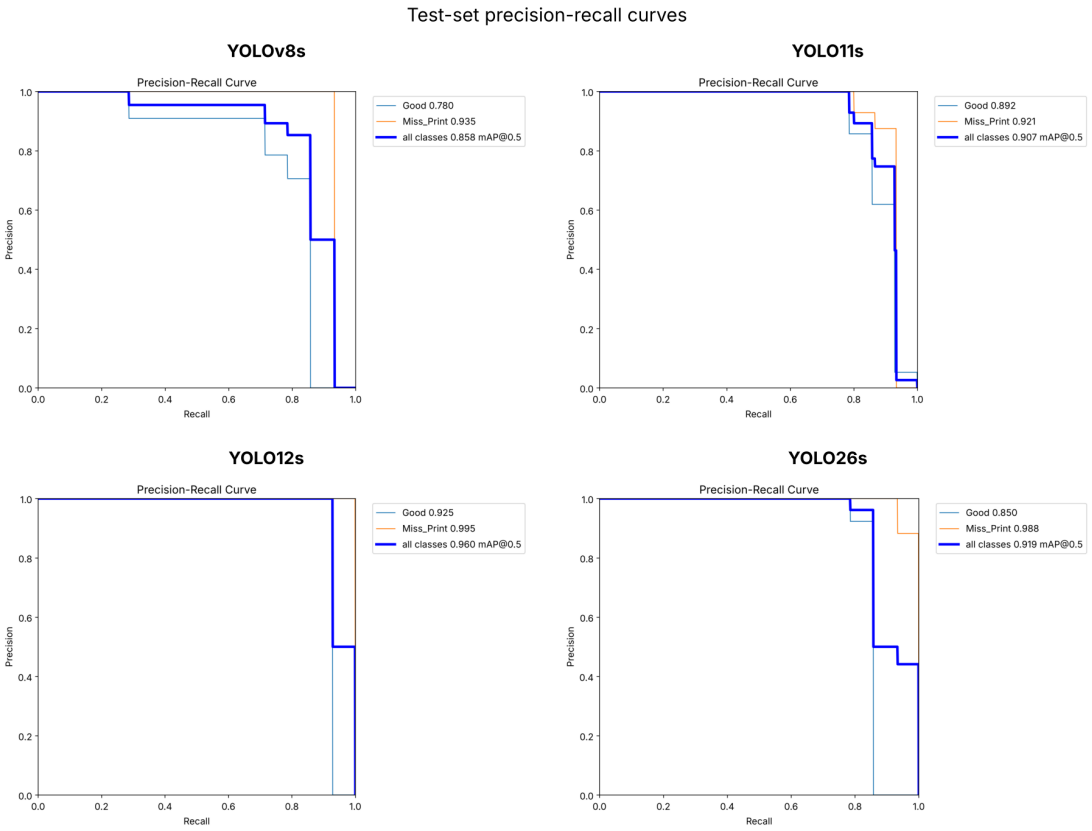


Figure 10: Precision–recall curves on the test split (re-generated on CPU with the same checkpoints).

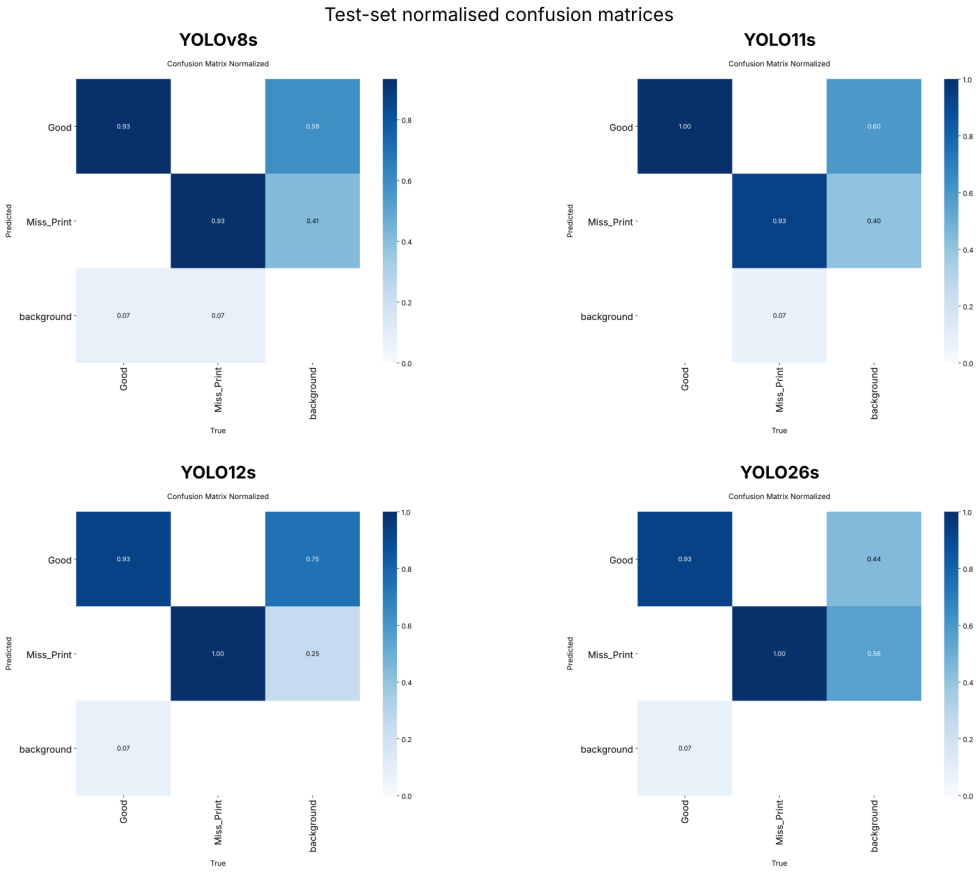


Figure 11: Normalised confusion matrices on the test split. The background column counts detections that match no ground-truth box and is sensitive to the confidence threshold used for the matrix.

C Source ledger

Items opened and read during the work, with what was taken from each. Entries marked "partly" could not be read in full.

- Ultralytics, tips for best training results (read): dataset-size guidance, epochs, pretrained weights [11].
- Ultralytics, YOLO26 versus YOLO11 (read): YOLO26 changes, parameters, FLOPs, COCO mAP [5].
- Chakrabarty, YOLO26 analysis (partly: first part verbatim, remainder via a model summary): architecture description [6].
- Tian et al., YOLOv12 (read): contributions, YOLOv12-S figures, FlashAttention limitation [4].
- Ultralytics, YOLO11 versus YOLOv8 (read): C3k2, C2PSA, parameters, COCO mAP [3].
- Zhang et al. (read abstract/page) [7]; He et al. (read full text) [9]; Bouthillier et al. (read) [10].
- Shorten and Khoshgoftaar (DOAJ record only; publisher PDF redirected) [8].
- Roboflow documentation, image augmentation (read): augmentation applied to training images only [12].
- Ultralytics configuration page (partly: patience, close_mosaic, optimiser defaults verified; augmentation defaults not verified, so none are quoted) [14].

- arXiv abstract pages for YOLO [2] and COCO [13]: authors and titles confirmed; venues are not stated on those pages and are omitted.

References

[1] Glenn Jocher, Jing Qiu, and Ayush Chaurasia. Ultralytics YOLO, 2023. Software, AGPL-3.0, https://github.com/ultralytics/ultralytics. Citation metadata version 8.0.0; package version 8.4.190 was used in this work.

[2] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection, 2015.

[3] Ultralytics. YOLO11 vs YOLOv8: a technical comparison, 2026. Documentation page, https://docs.ultralytics.com/compare/yolo11-vs-yolov8, accessed 10 November 2026.

[4] Yunjie Tian, Qixiang Ye, and David Doermann. YOLOv12: Attention-centric real-time object detectors, 2025. Technical report.

[5] Ultralytics. YOLO26 vs YOLO11, 2026. Documentation page, https://docs.ultralytics.com/compare/yolo26-vs-yolo11, accessed 10 November 2026.

[6] Sudip Chakrabarty. YOLO26: An analysis of NMS-free end to end framework for real-time object detection, 2026. Version 2. A secondary review based on Ultralytics documentation, not an independent benchmark.

[7] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization, 2017. Published at ICLR 2017.

[8] Connor Shorten and Taghi M. Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of Big Data*, 2019. Title, journal and year confirmed on the DOAJ record; author names and DOI taken from the publisher URL and search listing.

[9] Kaiming He, Ross Girshick, and Piotr Dollár. Rethinking ImageNet pre-training, 2018. ICCV 2019.

[10] Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Naz Sepah, Edward Raff, Kanika Madan, Vikram Voleti, Samira Ebrahimi Kahou, Vincent Michalski, Dmitriy Serdyuk, Tal Arbel, Chris Pal, Gaël Varoquaux, and Pascal Vincent. Accounting for variance in machine learning benchmarks, 2021.

[11] Ultralytics. Tips for best training results, 2026. YOLOv5 documentation page, https://docs.ultralytics.com/yolov5/tutorials/tips_for_best_training_results, accessed 10 November 2026.

[12] Roboflow. Image augmentation, 2026. Documentation page, https://docs.roboflow.com/datasets/image-augmentation, accessed 10 November 2026.

[13] Tsung-Yi Lin, Michael Maire, Serge Belongie, Lubomir Bourdev, Ross Girshick, James Hays, Pietro Perona, Deva Ramanan, C. Lawrence Zitnick, and Piotr Dollár. Microsoft COCO: Common objects in context, 2014.

[14] Ultralytics. Configuration (usage/cfg), 2026. Documentation page, https://docs.ultralytics.com/usage/cfg/, accessed 10 November 2026.